

ניהול זיכרון

מערכות הפעלה

זיכרון פיזי

זיכרון פיזי

- זיכרון פיזי הוא התקן חומרה שמאפשר שמירה של רצפים של אחדות ואפסים
- כל אחד מאיתנו מכיר התקני חומרה שמשמשים כזיכרון: לדוגמה הדיסק הקשיח, USB ודיסק אופטי
- כמו כן, הכרנו את זיכרון ה-Ram – יחידת הזיכרון – שמתוכו רצות תוכנות מחשב
- ייתכן ששמעתם גם על המושג "זיכרון Cache"
- מדוע יש צורך בסוגים שונים של זיכרונות ומה תפקידם?

זיכרון פיזי

• בתור משתמשים, ישנם ארבעה דברים שהיינו רוצים מזיכרון של מחשב:

1. **מהירות:** זמן הגישה (קריאה וכתיבה) של המעבד לזיכרון הוא בעל חשיבות קריטית לריצה מהירה של תוכניות - חישבו על מערכת דמיונית שבה יש מעבד מהיר, שמסוגל לבצע פעולת חיבור במיליארדית שניה, ואילו לצד המעבד יש זיכרון איטי שכל גישה אליו לוקחת שניה. פעולת חיבור של שני משתנים כוללת שתי גישות לזיכרון לטובת קריאת המשתנים, וקריאה נוספת לזיכרון לטובת כתיבת התוצאה. כמות הזמן הכללית שתיקח למערכת לבצע את החיבור תהיה לכן 3 שניות ועוד מיליארדית שניה... המהירות הרבה של המעבד פשוט לא באה לידי ביטוי עקב הזיכרון האיטי
2. **גודל:** האם קרה לכם שרציתם לשמור קובץ כלשהו על דיסק קשיח או על USB וקבלתם הודעת שגיאה שהדיסק מלא? נראה שאין צורך להסביר למה יותר זיכרון עדיף על פחות זיכרון (בהנחה שמשתנה רק גודל הזיכרון, ולא דברים אחרים כגון מהירות)
3. **יציבות:** אנחנו רוצים זיכרון שיאגור בתוכו את המידע גם כאשר הוא אינו מקבל חשמל. דמיינו שהדיסק הקשיח שלכם היה נמחק בכל פעם שהייתם מנתקים את המחשב מהחשמל, ושכל הדלקה של המחשב היתה כרוכה בהתקנה מחדש של מערכת ההפעלה.
4. **מחיר:** כמובן, עדיף לשלם כמה שפחות!

זיכרון פיזי

- בתור משתמשים, ישנם ארבעה דברים שהיינו רוצים מזיכרון של מחשב:

1. מהירות

2. גודל

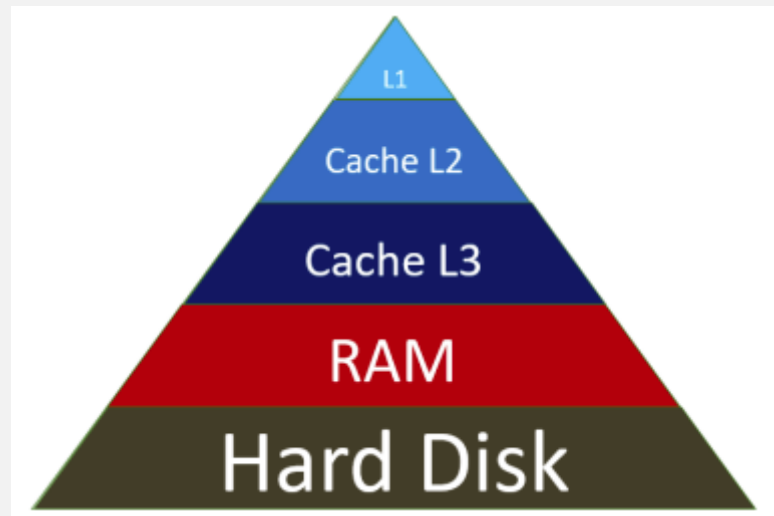
3. יציבות

4. מחיר

- הבעיה היא, שאין זיכרון שהוא אופטימלי מכל ארבעת הבחינות האלה
- זיכרון גדול יותר ומהיר יותר גם יהיה יקר יותר
- זיכרון קבוע יהיה יותר יקר מאשר זיכרון נדיף (זיכרון שנמחק עם הפסקת זרם החשמל, שנקרא Volatile לעומת זיכרון Non-Volatile, שנשמר גם ללא אספקת זרם חשמל)
- במילים אחרות, אין זיכרון אחד שעדיף על כל הזיכרונות. לכל זיכרון יש את נקודות העדיפות והנחיתות שלו על פני זיכרונות אחרים

זיכרון פיזי

- כדי לבנות מערכת בעלת ביצועים אופטימליים אנחנו צריכים לשלב סוגי זיכרונות שונים
- נשתמש בהרבה זיכרון איטי וזול, ומעט זיכרון מהיר ויקר
- התרשים הבא מייצג את ההיררכיה של התקני זיכרון- איטי וזול בתחתית הפירמידה, מהיר ויקר בקצה



זיכרון פיזי

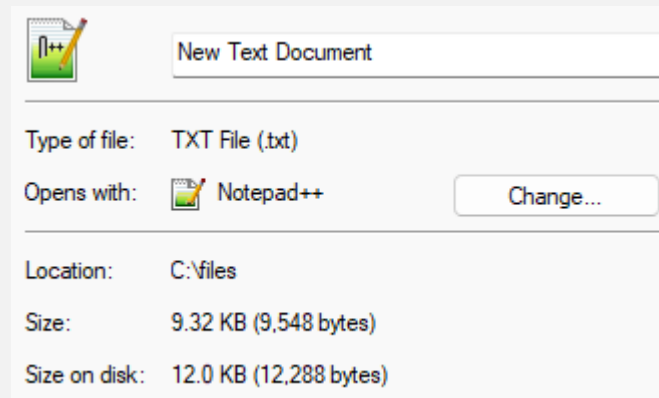
- התפקיד של מערכת ההפעלה הוא לנהל בצורה יעילה את משאבי הזיכרון שיש לה
- בתור כלל אצבע היינו רוצים שקוד שאנחנו זקוקים לו לעיתים קרובות יהיה בזיכרון מהיר, וככל שהצורך שלנו בקוד הוא לעיתים יותר רחוקות, כך נוכל להתפשר ולשמור אותו בזיכרון יותר איטי
- הבאת מידע מזיכרון איטי עלולה לקחת זמן רב בצורה ניכרת, כפי שמיד נראה, ולכן מערכת ההפעלה צריכה לנסות לצמצם ככל האפשר את הסיכוי שנצטרך מידע שנמצא בזיכרון איטי
- כמובן, שככל שיש לנו יותר זיכרון מהיר, כך עבודתה של מערכת ההפעלה קלה יותר והסיכוי שנמצא את מה שאנחנו צריכים בזיכרון המהיר הוא גדול יותר

זיכרון פיזי

- כדי לפשט מעט את התמונה, נתחיל בתיאור של מחשב שיש בו רק שני סוגים של זיכרון- דיסק קשיח ו-RAM
- הדיסק הקשיח הוא כלי קיבול זול וענקי, ב-250 שקל אפשר לרכוש דיסק קשיח בנפח של *terabyte*
 $1 \text{ terabyte} = 1024 \text{ gigabyte} = 1024^2 \text{ megabyte} = 1024^3 \text{ kilobyte} = 1024^4 \text{ byte} = 1,099,511,627,776 \text{ byte}$
- למעלה ממליון-מליוני בתים. דיסק כזה יכול לשמור בקלות מאות אלפי תמונות דיגיטליות
- הערה: יצרני הדיסקים נוטים לאמץ שיטת ספירה אחרת, בה הכפולות אינן של 1024 כפי שצריך להיות אלא של 1000 בלבד
- כך לדוגמה אם תרכשו דיסק בגודל 1TB תקבלו "רק" 1,000,000,000,000 בתים
- כפי שרואים, שיטת הספירה הזו מאפשרת ליצרנים למכור לכם 11% פחות מנפח הדיסק שחשבתם שאתם רוכשים...
- הדיסק הקשיח אינו נמחק כאשר הוא מנותק מחשמל, כלומר הוא Non-Volatile

זיכרון פיזי

- הדיסק הקשיח נשמר בכפולות של 4 קילו בתים, גודל שנקרא "בלוק"
- אם יש קובץ קטן מ-4kb הוא יישמר בבלוק שזה גודלו, ואם יש קובץ שגדול מבלוק יחיד, הגודל שלו יעוגל מעלה אל כפולת הבלוקים השלמה הקרובה
- לטובת ההמחשה, בחרו קובץ כלשהו ובידקו את ה- Properties שלו, תגלו שהגודל על הדיסק הוא כפולה של גודל בלוק:



זיכרון פיזי

- כיוון שזמן הגישה אל הדיסק הקשיח הוא כה גדול, המעבד אינו ניגש ישירות אל כתובות בזיכרון של הדיסק הקשיח
- במקום זאת, מערכת ההפעלה מפעילה Driver, שמביא בכל פעם לפחות בלוק אחד של מידע מהדיסק
- היתרון בכך שמביאים בלוק שלם מהדיסק הקשיח הוא שכמעט תמיד נצטרך להשתמש לא רק במידע שנמצא בבית מסויים אלא גם בבתים הקרובים לו
- כך, זמן הגישה הארוך אל הדיסק הקשיח מתחלק בין כל הבתים שנמצאים בבלוק
- לדוגמה, אם במערכת כלשהי גישה לדיסק הקשיח לוקחת 4 מילישניות, ובגישה הזו מובא בלוק שכאמור גודלו הוא 4kb (4096bytes), אז הבאה של כל בית מהדיסק הקשיח לוקחת בערך מיליונית שניה בלבד

זיכרון פיזי

- כיוון שאין למעבד יכולת לגשת לכתובת ספציפית בדיסק הקשיח, אלא רק לבלוק של מידע, המשמעות היא שאי אפשר להריץ תוכניות ישירות מהדיסק הקשיח
- הערה יש שיטה בשם Place In Execute שמאפשרת זאת בתנאים מסויימים, לא נתעסק בה
- זאת מכיוון שכאשר מעבד מריץ תוכנה, הוא חייב גישה לכתובות ספציפיות
- דמיינו מצב שבו המעבד מבקש להביא אליו את הפקודה הבאה להרצה, ובמקום לקבל פקודה יחידה הוא מקבל בלוק של 4096 בטים
- יש צורך בהתקן זיכרון אחר שמאפשר למעבד לגשת לכתובת ספציפית

זיכרון פיזי

- לעומת הדיסק הקשיח, גודלו של ה-RAM הוא קטן למדי ויקר הרבה יותר
- תמורת אותו מחיר שנשלם על דיסק קשיח ניתן לרכוש RAM שגודלו הוא רק אחוזים בודדים מגודלו של הדיסק הקשיח
- אם זה לא גרוע מספיק, ה-RAM הוא גם זיכרון מסוג Volatile, שיימחק בכל פעם שנכבה את המחשב
- היתרונות של ה-RAM על פני הדיסק הקשיח הם מהירותו והעובדה שהמעבד יכול לגשת באופן ישיר לכל כתובת ב-RAM
- זו הסיבה שכדי להריץ תוכניות ששמורות על הדיסק הקשיח, ראשית כל יש צורך להעתיק אותן אל ה-RAM

זיכרון פיזי

- אם נסכם עד כה, יש זיכרון ענק, איטי, ובלתי מחיק, אשר חלקים ממנו מועתקים לפי הצורך אל זיכרון קטן, מהיר ומחיק
- באופן זה אפשר לקבל נפח אחסון כמעט אינסופי, אך במקביל לדאוג שבזמן שהמעבד מריץ קוד יעמוד לרשותו זיכרון מהיר
- מנגנון ה-Cache הוא שיפור של מודל העבודה שסקרנו עד כה
- הרי לא כל המידע שמגיע ל-RAM משמש את המעבד בתכיפות זהה
- לדוגמה, נדמיין תוכנית מסויימת שמציגה למסך סרטון וידאו
- בעוד כל פריים בסרטון מוצג למסך פעם אחת בלבד, ייתכן שקטע קוד מסויים בתוכנית מורץ שוב ושוב
- ניקח את אותו קטע קוד ונשים אותו בזיכרון מהיר עוד יותר
- כך נוכל להאיץ את הגישה לזיכרון בדיוק לאותו אזור בזיכרון שהמעבד מרבה לפנות אליו

זיכרון פיזי

- כיצד יוצרים זיכרון מהיר יותר?
- לדוגמה, במקום לשים אותו על לוח האם (Motherboard) כמו את ה-RAM, יצרניות המעבדים שמות את ה-Cache ממש על הסיליקון של המעבד
- גם בסיליקון של המעבד יש אזורים "קרובים" יותר ליחידת העיבוד המרכזית, וכך נוצרים למעשה שלושה זיכרונות Cache, הידועים בכינויים L1, L2, L3 כאשר L1 הוא המהיר והקטן ביותר
- מצב שבו המעבד זקוק למידע מסוים והמידע הזה נמצא ב-Cache, נקרא Cache Hit
- המצב ההפוך, בו המידע אינו נמצא ב-Cache ויש להביא אותו מה-RAM או חס וחלילה מהדיסק הקשיח, נקרא Cache Miss
- חלק מהחוכמה שבקוד מערכת ההפעלה היא הלוגיקה של מתי להעביר מידע ל-Cache ומתי להחליף מידע שב-Cache במידע אחר

זיכרון פיזי

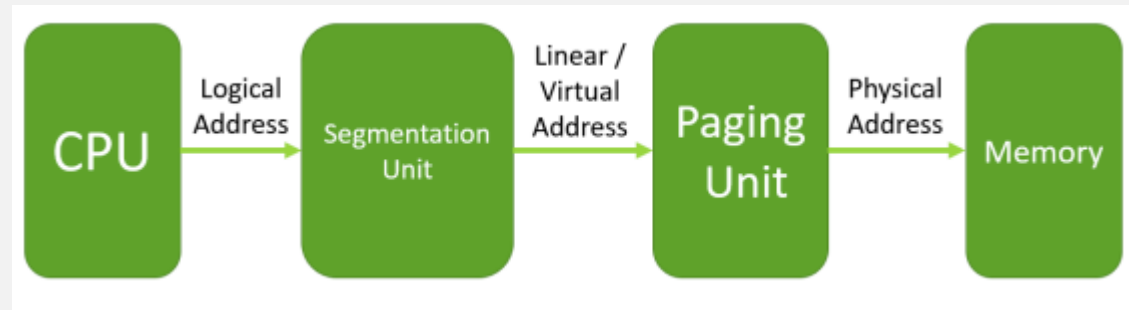
- עד כה דנו באופן תיאורטי בזיכרון מהיר יותר ופחות
- כדי להבין את סדרי הגודל המדוברים, נסקור את כמות הזמן שלוקחת גישה לזיכרון מסוגים שונים
- סדר גודל של זמני גישה:
 - רגיסטר – 0
 - L1 Cache – 0.5 ננו-שניות (ננו-שניה אחד חלקי מילארד שניה)
 - L2 Cache - 7 ננו-שניות
 - Ram – 100 ננו-שניות
- דיסק קשיח - זמן הגישה תלוי מאד בסוג הדיסק ובממשק הפיזי בינו לבין המעבד
 - דיסק קשיח מגנטי – סדר גודל של אלפית שניה
 - דיסק SSD – סדר גודל של 50,000 ננו-שניות

זיכרון פיזי

- נשווה זאת לחיים האמיתיים: אנחנו יושבים ליד שולחן כתיבה וזקוקים לעט
- פניה לרגיסטר היא כמו שימוש בעט שנמצא אצלנו ביד, אנחנו לא צריכים להקדיש זמן כדי להרים אותו
- גישה ל-L1 Cache היא כמו להרים את העט משולחן הכתיבה, נניח שפעולה זו לוקחת כחצי שניה
- גישה ל-L2 Cache היא כמו להוציא עט ממגירה שבשולחן הכתיבה, סדר גודל של מספר שניות
- גישה ל-RAM היא כמו לקום מהשולחן וללכת לקחת עט שנמצא בחדר אחר בבית, סדר גודל של דקה
- ומה לגבי גישה לדיסק הקשיח, בפרט אם הוא מגנטי? גישה זו, בהתאם ליחסי הזמנים שבדוגמאות הקודמות, היא כמו להקדיש כ- 251 שעות להשגת העט. כלומר גישה להארד דיסק נראית למעבד כמו איסוף של עט מחנות שנמצאת באוסטרליה
- גם אם נשתמש בדיסק SSD, זמן הגישה אליו מקביל לאיסוף העט מחנות שנמצאת במרחק שעה נסיעה.

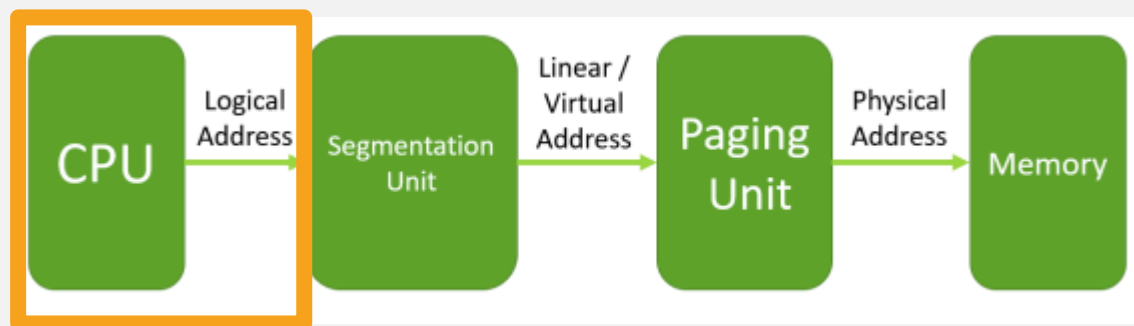
זיכרון פיזי

- כשסקרנו את מודל הארכיטקטורה, מתחנו קו שמחבר בין המעבד לבין הזיכרון
- זו הפשטה של המציאות. בפועל, החיבור בין המעבד לזיכרון הפיזי כולל כמה וכמה שלבים בדרך, לפי תרשים הזרימה הבא:



- נפרט ונסקור שלב שלב, מה הוא מבצע וכיצד.

זיכרון פיזי



```
mov eax, [ebx]
mov eax, [ebp]
jmp label
```

- שלב א: חלוקת הזיכרון ל-Segment-ים, כתובות לוגיות
- המעבד קיבל מספר פקודות אסמבלי לבצע:
- כל אחת מהפקודות הללו ניגשת לאזור זיכרון שונה:
- בפקודה הראשונה הרגיסטר `ebx` מצביע על משתנה בזיכרון הנתונים, הפקודה תטען לתוך `eax` את הערך שהמצביע מצביע עליו
- בפקודה השניה המעבד יזהה שהשימוש ברגיסטר `ebp` אומר שיש צורך לפנות אל אזור הזיכרון של המחסנית
- פקודת ה-`jmp` קובעת שיש לקפוץ למקום מסוים באזור הזיכרון בו נמצא קוד התוכנית

זיכרון פיזי

```
mov eax, [ebx]
mov eax, [ebp]
jmp label
```

- שלב א: חלוקת הזיכרון ל-Segment'ים, כתובות לוגיות

- המעבד קיבל מספר פקודות אסמבלי לבצע:

- כל אחד מאזורי הזיכרון נקרא Segment, ולמעבד יש רגיסטרים מיוחדים ששומרים את המיקום בזיכרון של כל ה-Segment'ים

- בפקודות האסמבלי שהובאו בתור דוגמאות נעשה שימוש ברגיסטרים הבאים:

- DS – Data Segment
- SS – Stack Segment
- CS – Code Segment

- כך שהדרך בה כתבנו את הפקודות היא למעשה קיצור כתיבה של הפקודות הללו:

```
mov eax, [DS:ebx]
mov eax, [SS:ebp]
jmp CS:label
```

זיכרון פיזי

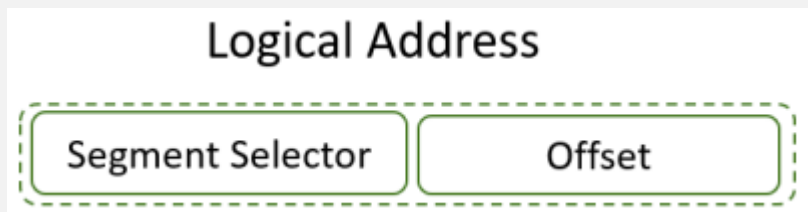
- צורת ההצגה הזו של כתובת נקראת כתובת לוגית, או Logical Address
- הכתובת הלוגית מחולקת ל-Segment ול-Offset, שהוא ההיסט מתחילת ה-Segment
- אם המונחים הללו נראים לכם מבלבלים, חישבו על ספרים
- אתם יכולים להחזיק מספר ספרים, בכל ספר יש עמודים רבים
- אם מישהו יבקש ממכם לפתוח ספר בעמוד 21, המידע לא יספיק לכם כדי לדעת לאיזה ספר הוא מתכוון
- לכן צריך קודם כל לציין על איזה ספר מדובר (ה-Segment) ולאחר מכן מה העמוד הספציפי (ההיסט מתחילת הספר)
- כל עוד מדובר בספרים ועמודים החישוב פשוט. אבל איך מתרגמים Segment והיסט לכתובת פיזית?

זיכרון פיזי

- בעבר היתה לכך שיטה פשוטה. במעבד 16 ביט צורת חישוב הכתובת הפיזית נקראת Real Mode
- בשיטה זו רגיסטר ה-Segment מחזיק את כתובת ההתחלה של ה-Segment, ויש לה שלושה שלבים פשוטים:
 1. הוצא את כתובת הבסיס של ה-Segment מתוך רגיסטר ה-Segment
 2. כפול את כתובת הבסיס של ה-Segment ב 16 (קל לבצע באמצעות הזזה של 4 ביטים שמאלה)
 3. הוסף לתוצאה את ההיסט
- דוגמה, אם רגיסטר ה-DS שווה לערך $2000h$ המשמעות היא ש-Segment הנתונים מתחיל בכתובת $2000h$ כפול 16 כלומר $20000h$
- אם נרצה להגיע להיסט מסוים פשוט נוסיף אותו לכתובת ההתחלה. לדוגמה, אם $bx=3012h$ אז השילוב $DS:bx$ יביא אותנו לכתובת $23012h$

זיכרון פיזי

- במעבדי 32 ביט, ומאוחר יותר גם 64 ביט, אימצו שיטת חישוב כתובות שנקראת Protected Mode והיא מורכבת הרבה יותר
- אך הבסיס לכל חישובי הכתובות עדיין נמצא בכתובת של ה-Segment ובהיסט בתוך ה-Segment
- מצד שני, ב-Protected Mode, כבר אי אפשר להוציא את כתובת הבסיס של ה-Segment ישירות מתוך רגיסטרי ה-Segment
- במקום זאת רגיסטרי ה-Segment כוללים אלא אוסף של שדות, כאשר אחד השדות הוא בגודל שני ביטים והוא מייצג את מעגל ההרשאה Protection Ring שה-Process שלנו נמצא בו
- לכן, החל ממערכות של 32 ביט כבר לא קוראים להם Segment Registers אלא Segment Selectors, כדי להדגיש שהמידע שהם שומרים השתנה



זיכרון פיזי



- **שלב ב: יחידת ה-Segmentation והמרה ל-Linear Address**
- סגמנטציה היא פעולה היסטורית במקצת, אחד התפקידים המרכזיים שלה - מניעה מ-Process לגשת לאזור זיכרון של Process אחר – מיושם כיום על ידי מנגנון ה-Paging שנדון בו בהמשך
- על כן, נסתפק בתיאור תמציתי יחסית של יחידת הסגמנטציה

זיכרון פיזי

- כיום נותרו ליחידת הסגמנטציה שני תפקידים:

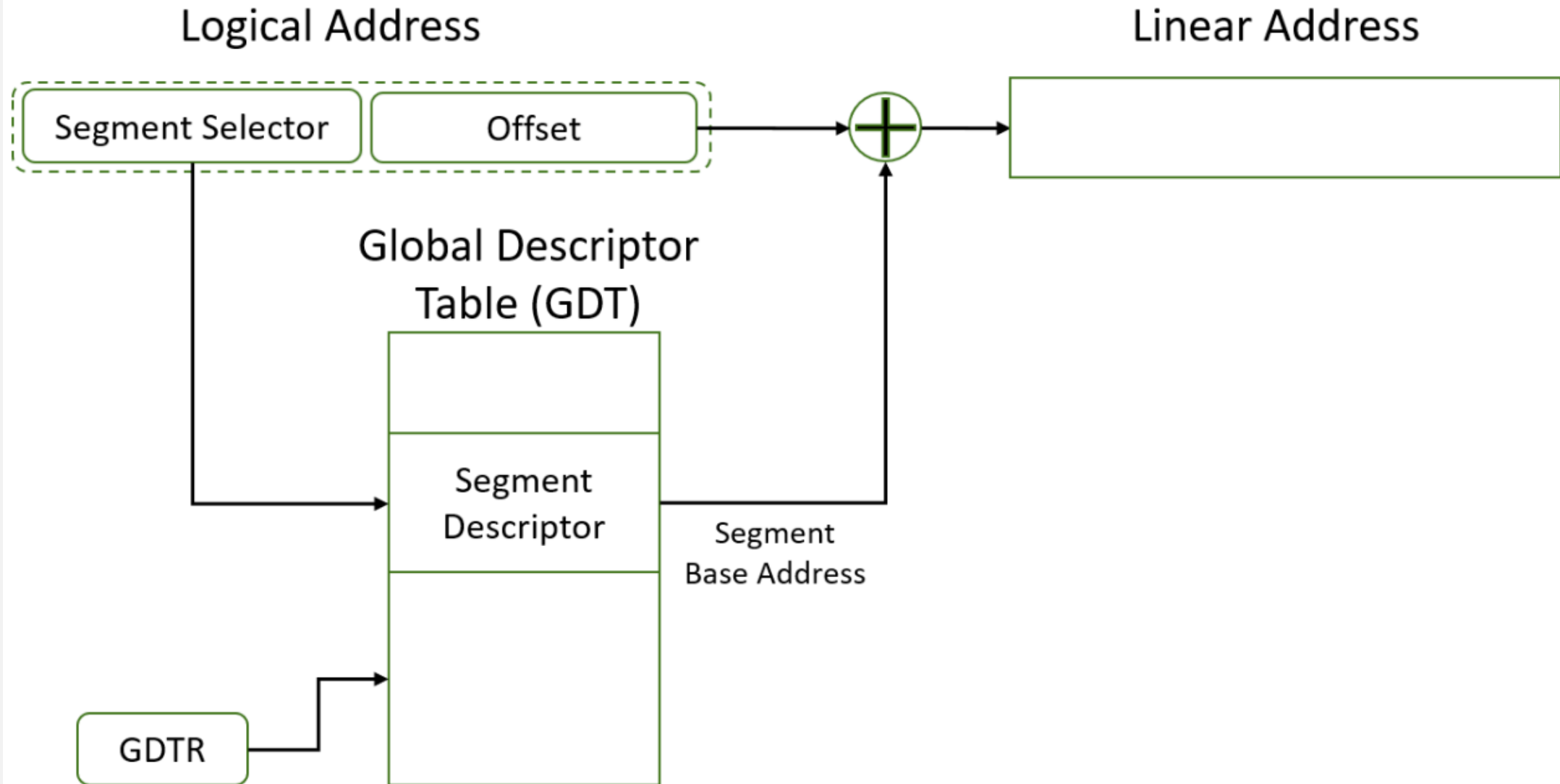
1. ליישם את מנגנון ה-Protection Rings וכך למנוע גישה לא מבוקרת של Thread לאזורי זיכרון שמעבר להרשאה שלו

2. לתרגם Logical Address ל-Linear Address. כתובת לינארית היא כתובת שאינה מבוססת על שילוב של Segment והיסט בתוך ה-Segment, אלא מספר יחיד שבאמצעותו אפשר לגשת אל הזיכרון הפיזי. שימו לב שכתובת לינארית אינה כתובת פיזית, יש ביניהן את מנגנון ה-Paging

זיכרון פיזי

- כל כתובת לוגית מורכבת כאמור מ-Effective Address ומ-Segment Selector
- ה Segment Selector מבציע לתוך טבלה שנקראת GDT (Global Descriptor Table)
- בתוך ה-GDT יש רשומות שמאפשרות למצוא את הכתובת שבה נמצאת כתובת הבסיס של ה-Segment
- זוכרים איך ב-Real Mode מוצאים את כתובת הבסיס של כל Segment פשוט בעזרת רגיסטר ה-Segment? בעצם מה שאנחנו עושים פה זה אותו דבר, רק שההמרה מתבצעת באמצעות טבלת חיפוש, שה-Segment Selector מצביע בה על הרשומה המתאימה. התוצאה של החיפוש בטבלה הזו היא כתובת הבסיס של ה-Segment
- לכתובת הבסיס של ה-Segment צריך להוסיף את ה-Effective Address וכך מגיעים אל הכתובת הלינארית
- מפאת חשיבות ה-GDT, למעבד יש רגיסטר מיוחד שנקרא GDTR שכל תפקידו הוא להחזיק את הכתובת של ה-GDT

זיכרון פיזי



זיכרון פיזי

- כעת הגענו בדיוק לנקודה שבזכותה קוראים ל- Protected Mode בשמו
- כאשר דנו ב- Segment Selectors הזכרנו שהם כוללים שני ביטים שמבטאים את ה- Protection Ring של ה-Process
- לכל רשומה ב-GDT צמוד מעגל ההרשאה שיש לה, מה שמאפשר למערכת ההפעלה להשוות בין ההרשאה של ה-Process לבין ההרשאה של Segment הקוד שהוא מנסה להריץ
- ההרשאה של ה-Process קרויה CPL (Current Protection Level)
- ההרשאה של הנדרשת קרויה DPL (Desired Protection Level)
- כיוון שכל גישה לזיכרון מתחילה מ- Segment Selector, אם Process מנסה לגשת למעגל הרשאה נמוך יותר מאשר מעגל ההרשאה שהוא נמצא בו, הגישה תיחסם ותתקבל שגיאה
- אם כך, איך מערכת ההפעלה מאפשרת לקוד לקרוא לפונקציות של הגרעין? בעזרת קריאות מערכת
- לפני שנמשיך לשלב הבא, אנו צריכים להכיר את המונח זיכרון ווירטואלי

זיכרון וירטואלי

זיכרון וירטואלי

- כדי להבין מדוע בכלל יש צורך בזיכרון וירטואלי, נסקור כיצד נראית מערכת המבוססת על מה שלמדנו עד כה

1. מרחב הזיכרון של כל Process נמצא ב-RAM

2. לכל Process יש מרחב זיכרון משלו

3. המעבד עובד ב-Protected Mode עם מעגלי הרשאה שונים

זיכרון וירטואלי

- כל אחד מהעקרונות הללו הוא נכון, אבל הם אינם מספיק טובים. בעיות לדוגמה:
- 1. **מרחב הזיכרון של כל Process נמצא ב-RAM**
- מה קורה אם ה-RAM קטן מלהכיל את מרחב הזיכרון של ה-Process?
- במערכות הפעלה של 32 ביט, ל-Process יש מרחב זיכרון של 4gb
- בעבר זיכרון RAM בגודל כזה היה לא מציאותי
- גם כיום, במערכות הפעלה של 64 ביט, אין RAM שגודלו 2^{64}
- במילים אחרות, קורה באופן שגרתי מצב שבו Process פונה לכתובת חוקית בזיכרון שלו, אולם אין כזו כתובת פיזית ב-RAM

זיכרון וירטואלי

- כל אחד מהעקרונות הללו הוא נכון, אבל הם אינם מספיק טובים. בעיות לדוגמה:

- **2. לכל Process יש מרחב זיכרון משלו**

- נניח שהצלחנו להכניס את כל מרחב הכתובות הפיזיות של Process אל תוך ה-RAM

- ברגע שיבוצע Context Switch אל Thread מתוך Process אחר, יהיה צריך להעביר ל-RAM את כל הזיכרון של ה-Process החדש

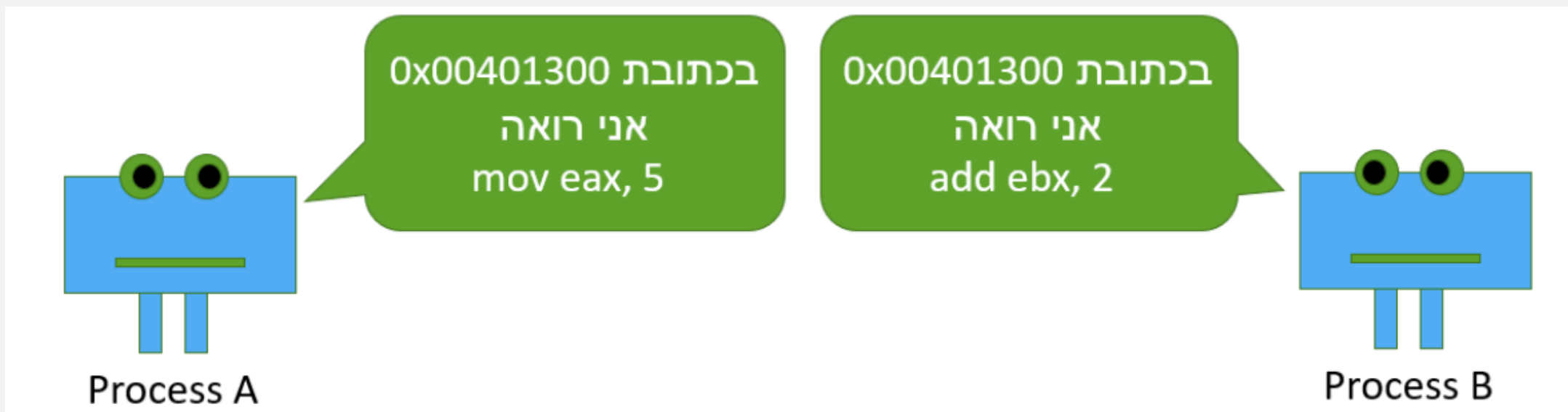
- כמובן שהחלפה של כל המידע ב-RAM לוקחת זמן משמעותי למדי ואינה פתרון הגיוני

זיכרון וירטואלי

- זיכרון וירטואלי הוא שיטה שמאפשרת להתגבר על הבעיות הללו
- הרעיון הכללי הוא שכאשר Process יפנה לכתובת הלינארית שנוצרה על ידי יחידת הסגמנטציה, זו לא תהיה כתובת פיזית אלא כתובת וירטואלית, שרק דרך מנגנון של תרגום כתובות ניתן להפוך לכתובת פיזית אליה המעבד ניגש
- נמחיש את הרעיון: הרגיסטר EIP מצביע על הפקודה הבאה בזיכרון, והוא מיוחס ל-Segment הקוד, כך שהכתובת הלוגית היא מסוג CS:EIP
- הכתובת הלוגית הזו מתורגמת על ידי יחידת הסגמנטציה לכתובת לינארית, נניח 0040|300h
- מה שה-Process כלל אינו מודע אליו, הוא שהוא לא באמת ניגש לזיכרון שבכתובת 0040|300h - זוהי כתובת וירטואלית
- במקום זאת, יש מנגנון שמחליף את הכתובת 0040|300h בכתובת אחרת, נניח 07|45300h
- איך מתבצעת ההחלפה? ואיך מערכת ההפעלה דואגת לכך שלאחר ההחלפה ה-Process אכן יגיע לזיכרון הפיזי שהוא חושב שהוא נמצא בו? מי שמבצע זאת הוא מנגנון ה-Paging

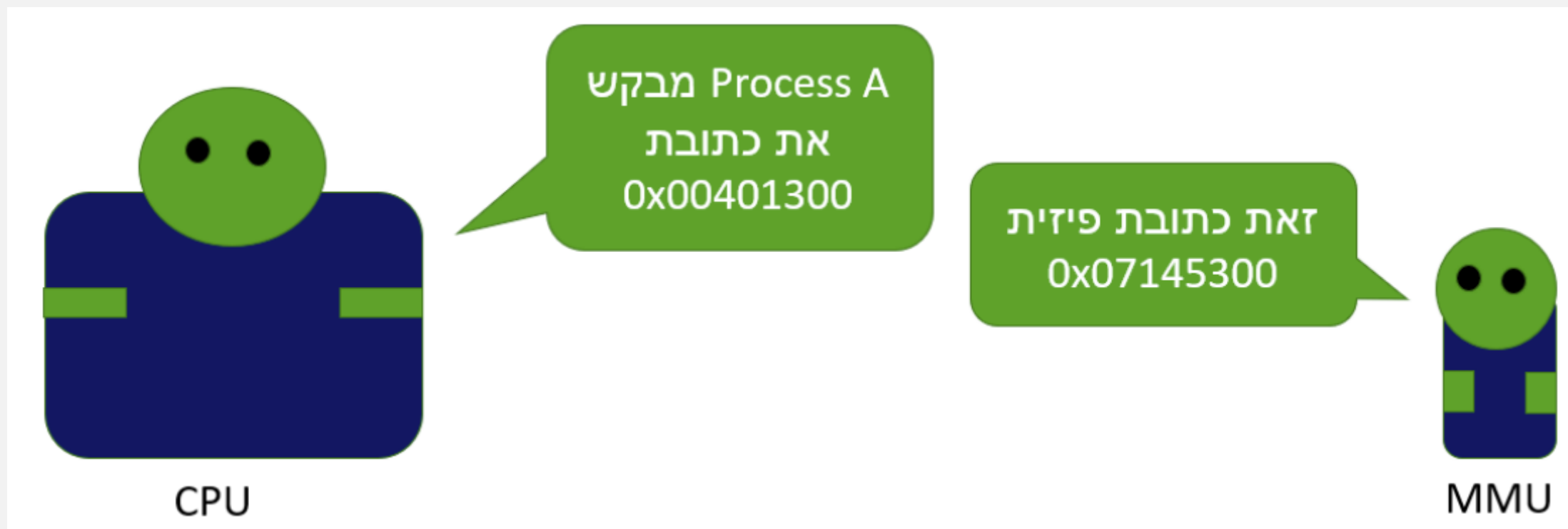
זיכרון וירטואלי

- הבסיס של כתובות וירטואליות הוא שלכל Process יש מרחב כתובות וירטואליות משלו
- מבחינת כל Process כל הכתובות בזיכרון שייכות לו ורק לו
- ה- Process לא מודע לכך שאותה הכתובת הוירטואלית קיימת גם עבור Process אחר, ומערכת ההפעלה מבצעת המרות והחלפות כתובות מאחרי הקלעים. כך לדוגמה, אם יש שני Processים על המחשב, לשניהם תהיה כתובת וירטואלית 00401300h אך יש סיכוי גבוה שכל Process יראה בכתובת הזו מידע שונה, מכיוון שלכל Process מבוצע תרגום אחר בין הכתובות הוירטואליות לכתובות הפיזיות

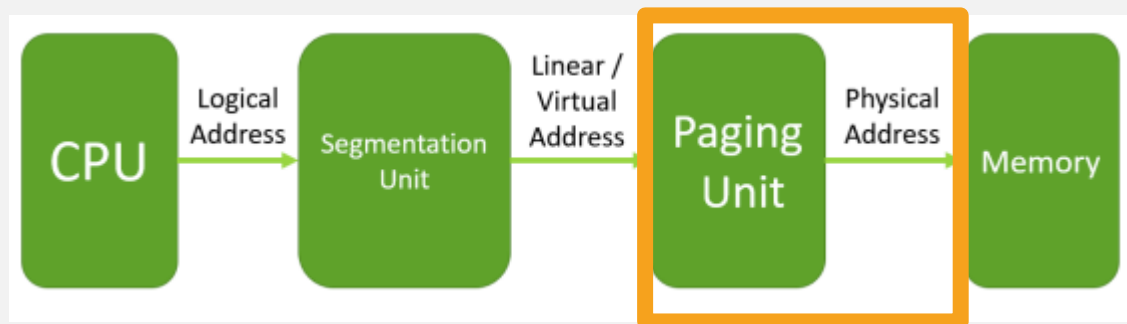


זיכרון וירטואלי

- ההמרה בין כתובת וירטואלית ופיזית מתבצעת באמצעות רכיב בשם MMU (Memory Management Unit)
- כאשר ה-CPU נתקל בכתובת וירטואלית כלשהי שעליו לגשת אליה, הוא מעביר את הכתובת לפענוח ב-MMU

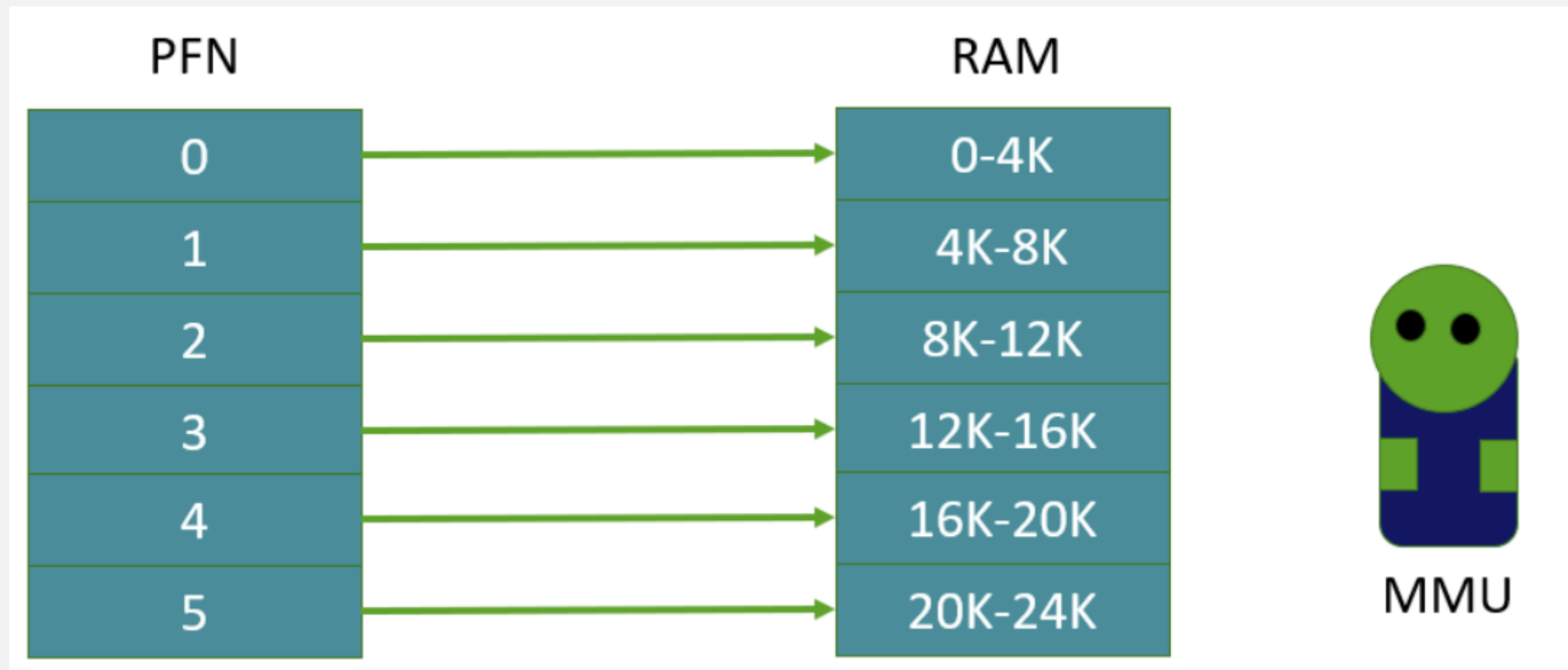


זיכרון וירטואלי



- שלב ג: יחידת ה-Paging והמרה ל-Physical Address
- ה-MMU מחלק את הזיכרון הפיזי למקטעים
- כל מקטע נקרא Page Frame (או פשוט Frame)
- Frames יכולים להיות בגדלים שונים, חזקות של 2 (אפשרויות נפוצות: 4kb, 2mb, 4mb)
- אנחנו נשתמש בדיון שלנו ב-Frames בגודל הנפוץ של 4kb
- כל Frame מקבל מספר מזהה שנקרא PFN (Page Frame Number)

זיכרון וירטואלי



זיכרון וירטואלי

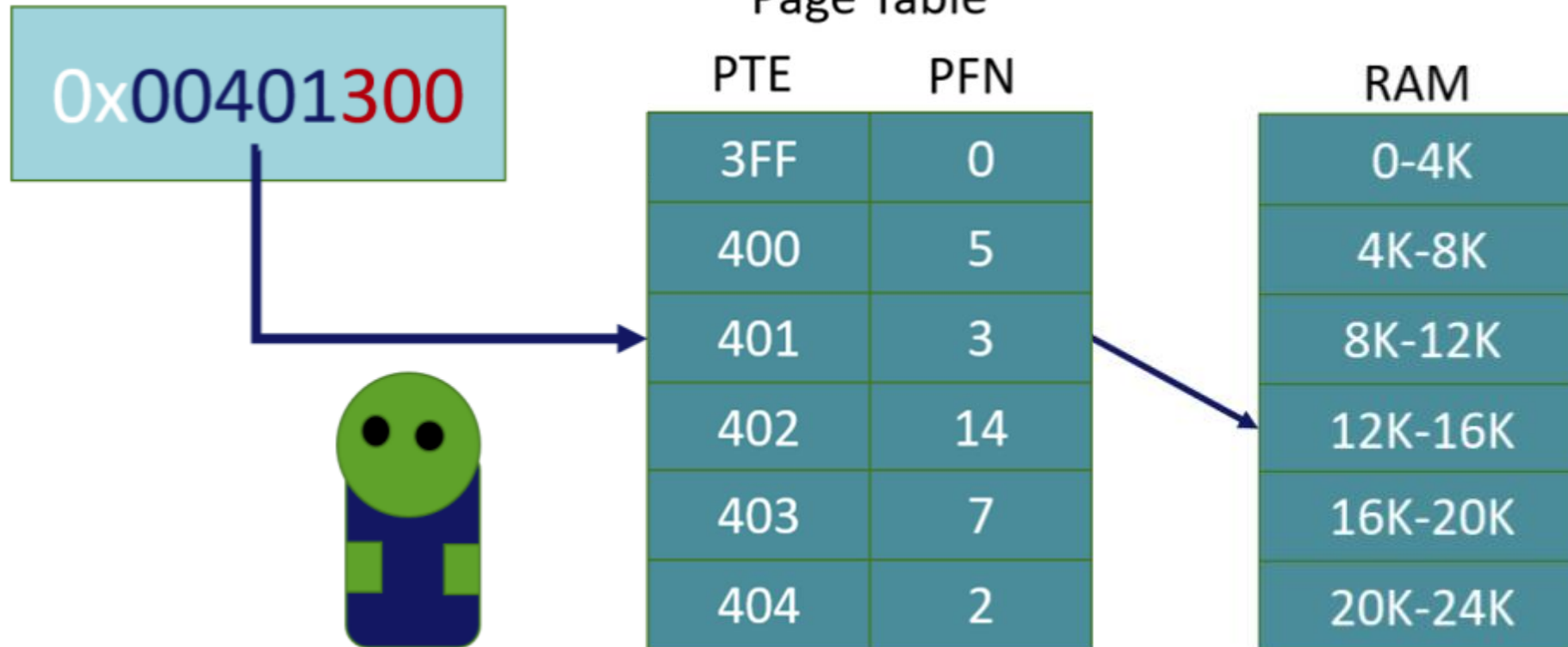
- ל-MMU יש טבלה, שמאפשרת לו להמיר בקלות כתובת וירטואלית לכתובת פיזית
- כל שורה בטבלה כוללת PFN שמצביע על זיכרון ה-RAM
- PTE (Page Table Entry) שהוא פשוט מספר השורה בטבלה

Page Table	
PTE	PFN
3FF	0
400	5
401	3
402	14
403	7
404	2

זיכרון וירטואלי

- נחלק את הכתובת הוירטואלית לשני חלקים: החלק הראשון ישמש אותנו כדי להכנס למקום הנכון בטבלה, וכך להגיע ל-Frame הנכון ב-RAM
- החלק השני יהיה כמה בתים אנחנו צריכים להוסיף מתחילת ה-Frame כדי להגיע בדיוק לכתובת המבוקשת

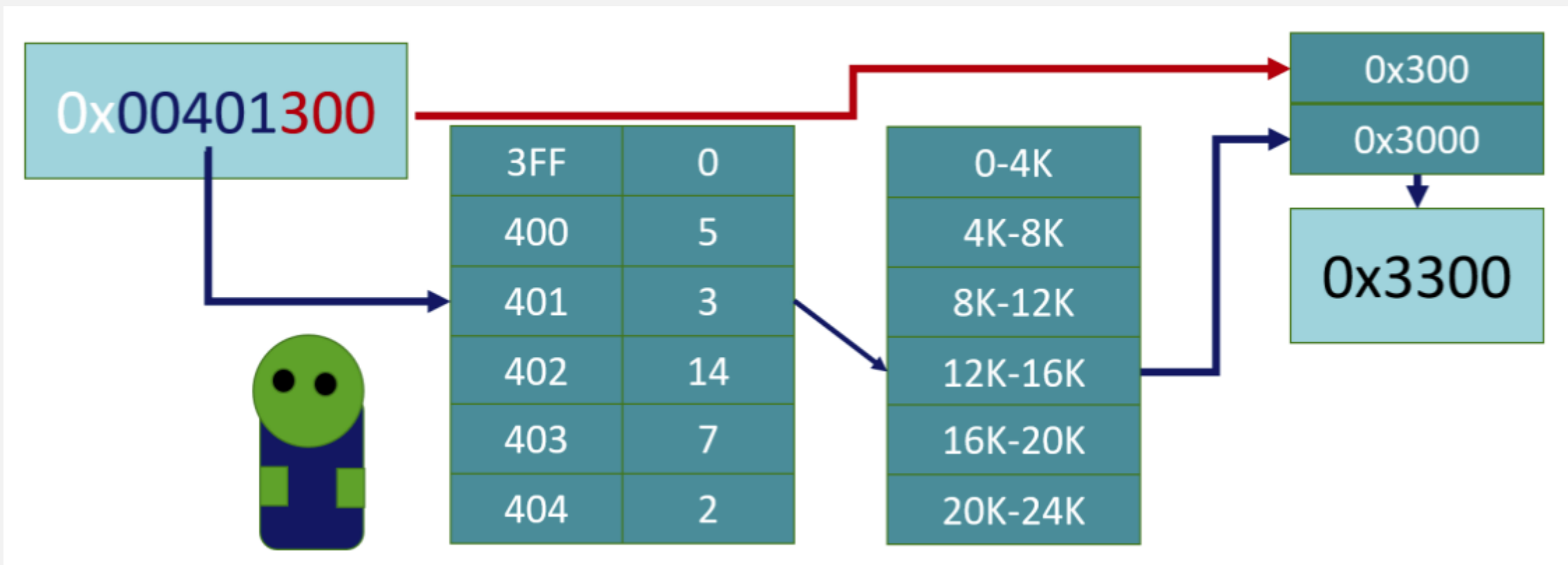
- המחשה:



$$\begin{aligned} 12 \times 1024 &= 12288 \\ &= 3000h \\ 3000h + 300h \\ &= 3300h \end{aligned}$$

- לכן הכתובת הפיזית שנגיע אליה היא $3300h$

זיכרון וירטואלי



זיכרון וירטואלי

- מדוע הכתובת הוירטואלית מחולקת דווקא ל-5 ספרות הקסדצימליות (20 ביטים) ל PTE ו-3 ספרות (12 ביטים) להיסט בתוך ה Frame?

0x00401300

- נזכור שהנחת המוצא שלנו היא שבחרנו באפשרות שבה גודל Frame הוא 4096byye (בהקסהדצימלי 1000h) כך שההיסטים האפשריים בתוך Frame הם מ-000h עד FFFh

זיכרון וירטואלי

- לשיטה שסקרנו עד כה להמרת כתובות וירטואליות לפיזיות יש בעיה
- מה גודל ה Page Table שצריך ה-MMU לטובת ביצוע ההמרות?
- בכתובת הוירטואלית מוקצים לטובת ההיסט 12 ביט, לכן במערכת של 32 ביט יוותרו 20 ביטים שמבטאים את ה-PTE
- ה-20 ביטים הללו מיתרגמים ל 2^{20} כניסות לטבלה - למעלה ממיליון כניסות
- כלומר ה-MMU צריך ליצור טבלה בת מיליון רשומות רק לטובת ההמרה. טבלה זו צריכה להשתנות עבור כל Process
- נקבל בזבז עצום של זיכרון, רק לטובת שמירת ה Page Tables של כל ה-Processes
- במערכות של 64 ביט גודל ה Page Table יהיה כה עצום שהוא עלול להיות יותר גדול מהזיכרון הפיזי שעומד לרשותנו כדי לשמור אותו

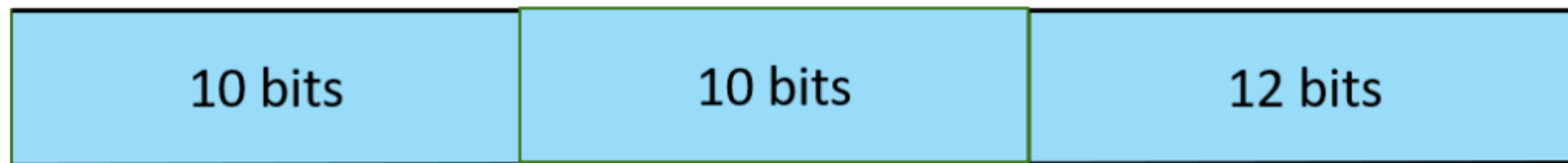
זיכרון וירטואלי

- כדי למנוע את הבעיה הזו, במקום שיהיה לנו Table Page אחד ענק, ניצור הרבה Page Tables קטנים יותר
- לכאורה הרבה טבלאות קטנות תופסות את אותו מקום בזיכרון של טבלה אחת גדולה ולכן לא חסכנו זיכרון, אבל למעשה ברוב המקרים ל-Process יש מרחבי זיכרון לא מנוצלים ורק לעיתים נדירות Process תופס את כל הזיכרון הוירטואלי שעומד לרשותו
- התכונה הזו מאפשרת לנו ליצור עבור Process חדש Page Table אחד קטן, ואם ה-Process צריך עוד זיכרון נוסיף לו עוד שנמלא את דרישות הזיכרון שלו
- עת כשיש לנו הרבה Page Tables קטנים, מספרי ה-PTE צפויים לחזור על עצמם
- לדוגמה ב- Page Table מספר 0, ה-PTE שמספרו 401h יכול להצביע על זיכרון פיזי שמתחיל בכתובת 00003000h ואילו ב Page Table מספר 1, ה-PTE שמספרו הוא גם כן 401h יצביע על כתובת אחרת לגמרי בזיכרון הפיזי, לדוגמה 02100000h
- לכן מערכת ההפעלה צריכה לדעת לאיזה Page Table בדיוק מתייחס ה-PTE שיש בכתובת הוירטואלית, ובשביל זה יש לה טבלה נוספת: Page Directory

זיכרון וירטואלי

- ה Page Directory קובעת לאיזה Page Table צריך להגיע כדי לפענח את הכתובת ה-PTE
- כך נראית החלוקה עבור כתובות של 32 ביט:

32 bit virtual address

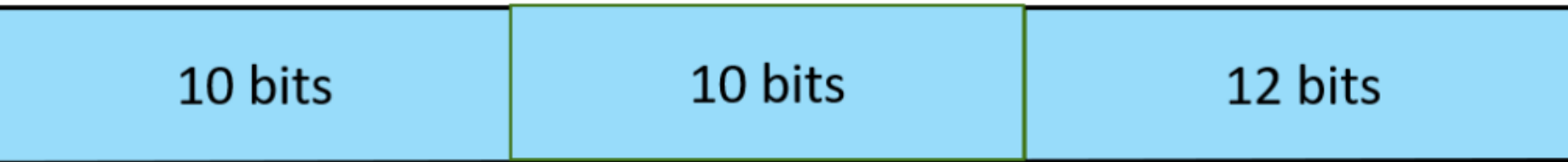


באיזו Page Table
נשתמש

איזו Page Table
Entry ניקח

ה-offset מתחילת
ה-Page Frame

32 bit virtual address



באיזו Page Table
נשתמש

איזו Page Table
Entry ניקח

ה-offset מתחילת
ה-Page Frame

- לדוגמא
- הכתובת הווירטואלית
- 01234567h

0000 0001 0010 0011 0100 0101 0110 0111

Page Directory Entry – 0000 0001 00 = 0x4

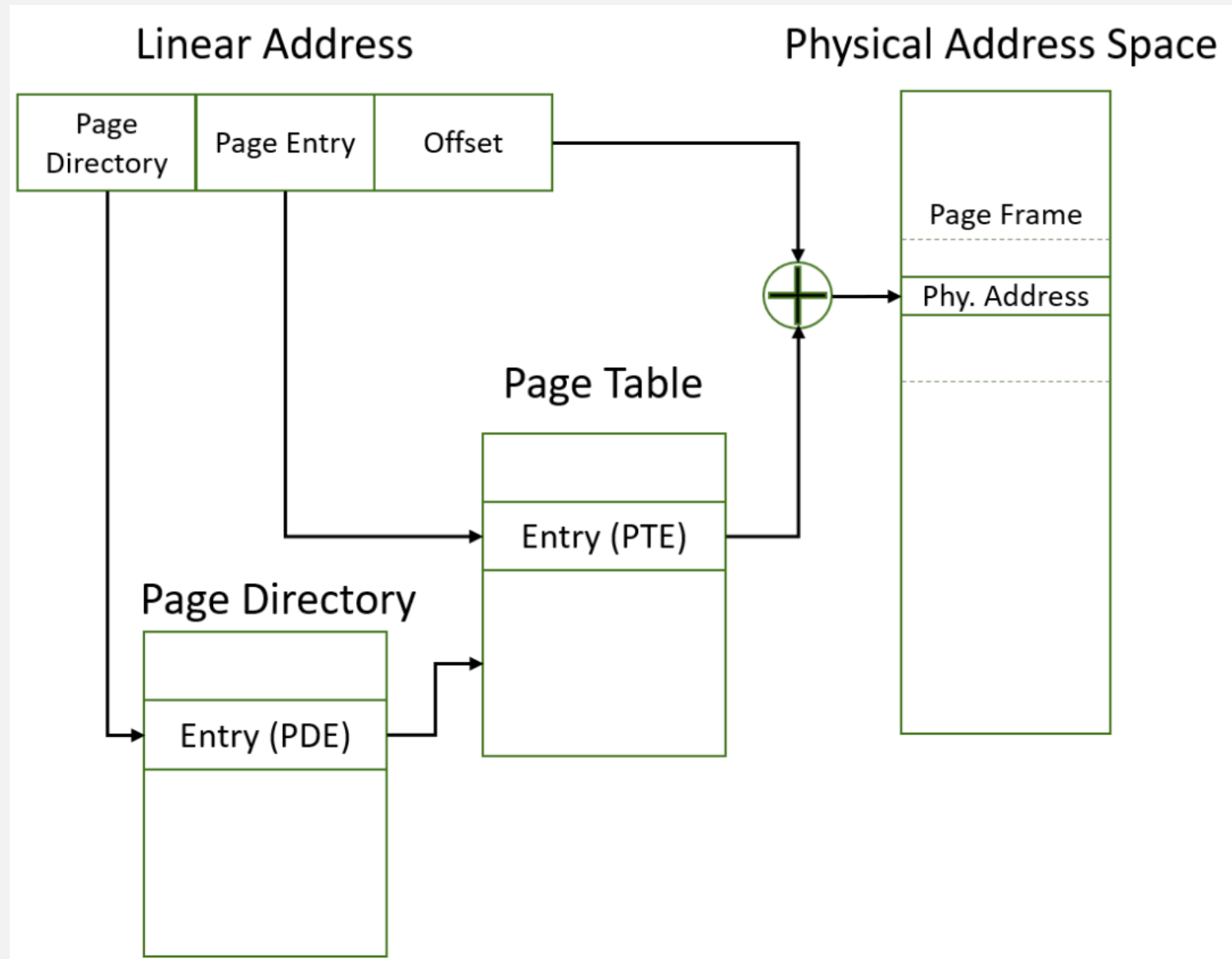
Page Table Entry – 10 0011 0100 = 0x234

Page Frame Offset – 0101 0110 0111 = 0x567

- כלומר:

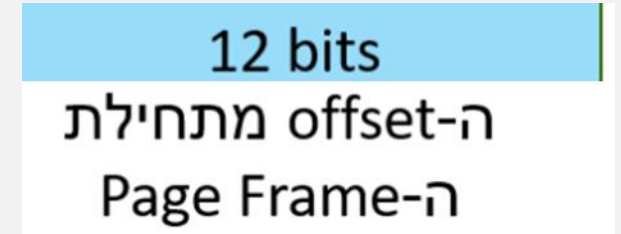
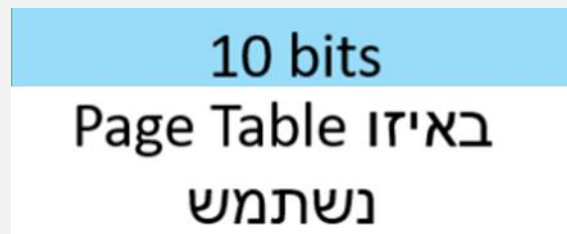
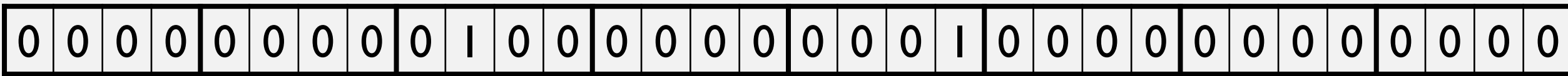
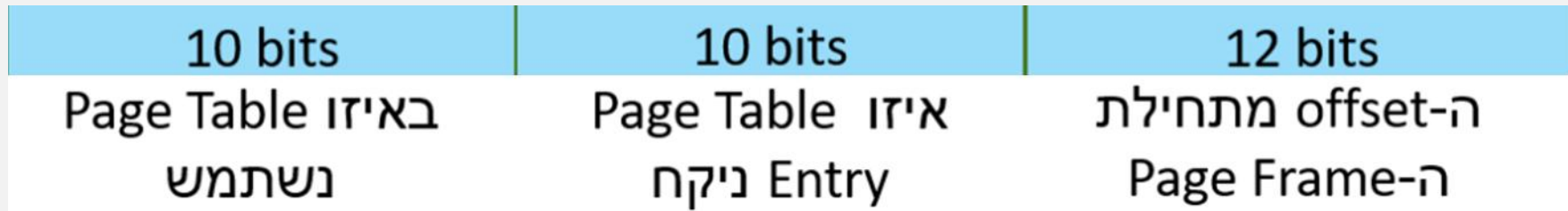
- יש לגשת לרשומה 4 ב-Page Directory - שם נראה מה הכתובת של ה-Page Table
- ב-Page Table נלך לרשומה 234h לקבל את ההתחלת ה-Frame ונוסיף לכתובת זו 567h

זיכרון וירטואלי



זיכרון וירטואלי

- במקרים רבים כשחוקרים תוכנה מגלים שהקוד שלה מתחיל בכתובת וירטואלית 401000h
- זוהי כתובת ברירת המחדל לתחילת אזור הקוד בקבצי תוכנה שונים. מה מיוחד דווקא בכתובת זו?
- בואו נפרק אותה. כיוון שזו כתובת של 32 ביטים נשלים עם שני 0 - 00401000



- כלומר: כברירת מחדל נלך לכניסה הראשונה בDirectory, ומהטבלה שנגיע אליה נלך גם לראשונה

זיכרון וירטואלי

- **חילוף הקשר בין תהליכים**
- כפי שתיארנו, במערכת ההפעלה תהליכים רצים באופן מקבילי: כל פעם רץ תהליך לזמן קצר ואז נית השליטה לתהליך אחר
- המעבר בין התהליכים נקרא חילוף הקשר (Context Switch)
- נראה כיצד חילוף הקשר נכנס לתמונה
- לשם כך נכיר רגיסטר בשם CR3

זיכרון וירטואלי

- במעבדי אינטל 32 ביט ישנם שמונה רגיסטרי בקרה הנקראים CR (Control Registers) הממוספרים מ 0 עד 7 (מעבדי 64 כוללים גם CR8)
- רגיסטרים אלו שולטים אלו על מגוון פעולות הקשורות להפעלה של המעבד עצמו. לדוגמה:
- משפחת מעבדי אינטל ידועה בכך שיש תאימות לאחור בין דורות המעבדים שלה. כלומר, גם מעבד מודרני יכול להריץ תוכנות שנכתבו עבור מעבד מוקדם. כפי שהזכרנו, מעבדים ישנים תמכו ב Real Mode. מעבדים מודרניים תומכים ב Protected Mode. גם כיום, כאשר מחשב נדלק, המעבדים שלו עולים קודם כל ב Real Mode ורק לאחר מכן עוברים ל Protected. פעולת הבקרה הזו מבוצעת על ידי הביט הראשון ברגיסטר CR0
- כשהכרנו את מנכנון Page Frame הנחנו שגודלו הוא 4kb, אך גודל Page יכול להיות שונה. הביט האחרון ב CR0 קובע האם ניתן לשנות את הגודל

זיכרון וירטואלי

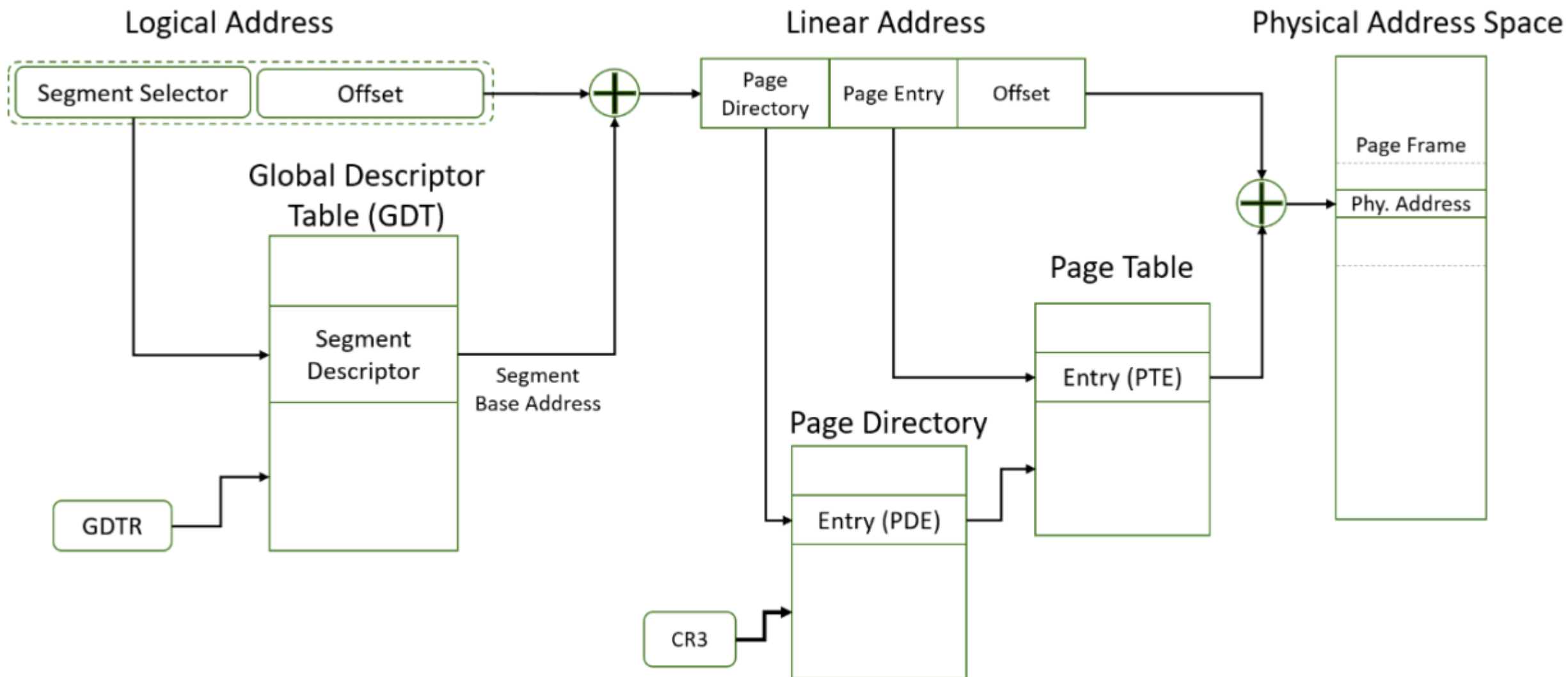
- אחד מהרגיסטרים הללו, CR3, כולל שדה בשם Page Directory Base Address.
- זוהי הכתובת של ה-Page Directory - היכן שמתחיל כל תהליך התרגום מכתובת לינארית (וירטואלית) לכתובת פיזית
- כל מה שצריך לעשות כדי ליצור תרגום שונה לגמרי הוא רק להחליף את ה-Page Directory שמשמש לתרגום
- הכתובת ב-CR3 היא כתובת פיזית - כל תהליך תרגום כתובת וירטואלית לפיזית מתחיל מהכתובת הזו. לכן הכתובת הזו היתה וירטואלית, המצב שהיה נוצר הוא שהכתובת ב-CR3 מתרגמת את עצמה
- למערכת ההפעלה יש Page Directory עבור כל תהליך שהיא מריצה, בכל פעם שתהליך חדש נוצר, נוצר עבורו Page Directory שיאפשר לתהליך לגשת לאזורים שמערכת ההפעלה הקצתה לו ב-RAM

זיכרון וירטואלי

- בעת ביצוע חילוף הקשר, מערכת ההפעלה מבצעת החלפה של הרגיסטרים של המעבד. בעקבות ההחלפה יתרחשו שני דברים:
 1. הרגיסטר EIP שמחזיק את מיקום הפקודה הבאה יצביע על המקום בו התהליך עצר את ריצתו בפעם האחרונה שמערכת ההפעלה נתנה לו זמן מעבד
 2. הרגיסטר CR3 שמחזיק את המפתח לכל הזיכרון של התהליך יעבור להצביע על דפי הזיכרון של התהליך החדש
- כך אין צורך "להחליף" את הזיכרון הפיזי ב-RAM, דבר שיכול לקחת זמן ניכר. מספיק רק להחליף רגיסטר אחד

זיכרון וירטואלי

- לסיכום: התמונה המלאה של תהליך המרת הכתובות, מכתובת לוגית (וירטואלית) לכתובת פיזית



זיכרון וירטואלי

• תפקידים נוספים של של Page Table וPage Directory

- כפי שאפשר לראות באיור הבא, בנוסף לחלק שלהם בפענוח כתובות וירטואליות, לטבלאות יש גם תפקידים נוספים. לכל רשומה שם מתווספים כמה שדות מידע מעניינים

32-Bit Page Directory Entry (PDE)

20 bit page table base address	Avail	G	PS	0	A	PCD	PWT	U/S	W	P
--------------------------------	-------	---	----	---	---	-----	-----	-----	---	---

32-Bit Page Table Entry (PTE)

20 bit page base address	Avail	G	PAT	D	A	PCD	PWT	U/S	W	P
--------------------------	-------	---	-----	---	---	-----	-----	-----	---	---

- נכיר שניים מהשדות

זיכרון וירטואלי

32-Bit Page Directory Entry (PDE)

20 bit page table base address	Avail	G	PS	0	A	PCD	PWT	U/S	W	P
--------------------------------	-------	---	----	---	---	-----	-----	-----	---	---

32-Bit Page Table Entry (PTE)

20 bit page base address	Avail	G	PAT	D	A	PCD	PWT	U/S	W	P
--------------------------	-------	---	-----	---	---	-----	-----	-----	---	---

- שדה U/S (User/Supervisor) כולל ביט אחד שקובע האם הזכרון דורש הרשאת גרעין (0) או הרשאות חלשות יותר (1)
- שדה W (Write) מציין האם ניתן לכתוב לזיכרון, או שהוא לקריאה בלבד